

From Decoder-Only to Encoder-Decoder: Repurposing LLMs for Abstractive Summarization

Giang-Bien Kieu

Hanoi University of Science and Technology Viettel Artificial Intelligence Center
HUST, Vietnam
bienkieugiang@gmail.com

Xuan-Dung Doan

Viettel Group, Vietnam
dungdx4@viettel.com.vn

Abstract

Decoder-only large language models are strong generators, but they are not a natural fit for long-input summarization. They read and write through one causal stream. This makes them expensive when the input is long and the output must stay faithful. We present LLM2Seq, an encoder-decoder framework for abstractive summarization. LLM2Seq reuses a decoder-only LLM as a bidirectional encoder, connects it to a lightweight Transformer decoder through a gated residual adaptor, and optionally adds multi-token prediction (MTP) for faster decoding. We evaluate on WikiLingua with 3,901 test examples. Using a Qwen encoder, LLM2Seq reaches 54.01 ROUGE-1, 20.83 ROUGE-2, and 31.83 ROUGE-L. This is close to a larger T5Gemma-2-1B-1B LoRA baseline on ROUGE-1, while producing shorter summaries. MTP with verified decoding gives a 1.17x measured token-per-second speedup for the Qwen variant with only small ROUGE changes. The result shows that decoder-only LLMs can be reused for summarization, but decoding speedup depends on system overhead, not only on accepted tokens per step.

1 Introduction

Summarization is easy to state. A system reads a long document and writes a shorter one. The hard part is deciding what to keep. The model must compress the text, preserve facts, and generate fluent output.

Decoder-only LLMs are now the dominant form of large language model (Brown et al., 2020; Dubey et al., 2024; Yang et al., 2024). They scale well and generate strong text. But their causal structure is not ideal for sequence-to-sequence tasks. For summarization, the source document should be read first. Then the model should generate the summary.

Encoder-decoder models follow this pattern naturally (Sutskever et al., 2014; Bahdanau et al., 2015; Vaswani et al., 2017). The encoder builds a source representation. The decoder generates from it using cross-attention. This design is used by T5, BART, and mT5 (Raffel et al., 2020; Lewis et al., 2020; Xue et al., 2021). The problem is that many strong recent LLMs are decoder-only. If we want their representations inside an encoder-decoder model, we need to repurpose them.

LLM2Seq starts from that idea. It uses an LLM2Vec-style encoder (BehnamGhader et al., 2024). It then adds a gated residual adaptor and a lightweight decoder. The adaptor is important. It must turn the encoder memory into something the decoder can use without losing source information.

The project is a summarization study. We train and evaluate on WikiLingua (Ladhak et al., 2020). We compare LLM2Seq with Llama and Qwen encoders against a T5Gemma-2-1B-1B LoRA baseline (Zhang et al., 2025). We also test MTP with self-distillation for faster decoding.

Our main findings are:

- LLM2Seq with a Qwen encoder is much stronger than the Llama encoder on WikiLingua.
- T5Gemma gives the best ROUGE-2 and ROUGE-L, but it produces much longer summaries.
- Verified MTP keeps quality nearly stable for Qwen and gives 1.17x measured decoding speedup.
- MTP does not guarantee speedup. The Llama variant accepts more tokens per step but still becomes slower because verification overhead dominates.

2 Related Work

Encoder-decoder summarization. Sequence-to-sequence learning first used recurrent encoders and decoders (Sutskever et al., 2014). Attention improved this by letting the decoder focus on source states (Bahdanau et al., 2015). The Transformer made self-attention the standard architecture (Vaswani et al., 2017). T5 frames every task as text-to-text generation (Raffel et al., 2020). BART uses denoising pretraining and works well for generation and summarization (Lewis et al., 2020). mT5 extends this recipe to multilingual data (Xue et al., 2021).

Decoder-only LLMs as encoders. LLM2Vec shows that a decoder-only LLM can become a strong bidirectional text encoder (BehnamGhader et al., 2024). GRIT also shows that generative models can learn useful representations for retrieval and embedding tasks (Muennighoff et al., 2024). LaMaTE applies a related idea to machine translation, using LLMs as encoders for seq2seq generation (Luo et al., 2025). T5Gemma adapts Gemma-style models into encoder-decoder models (Zhang et al., 2025). LLM2Seq follows the same direction, but focuses on abstractive summarization.

Efficient decoding. Speculative decoding drafts tokens and verifies them with a target model (Leviathan et al., 2023). Blockwise parallel decoding predicts multiple future tokens in parallel (Stern et al., 2018). Medusa adds extra heads to accelerate LLM inference (Cai et al., 2024). Multi-token prediction trains models to predict future tokens directly (Gloeckle et al., 2024). MTP-D improves MTP heads with self-distillation (Zhao et al., 2026). The distillation part follows the broader idea of transferring information from a teacher distribution to a student model (Hinton et al., 2015). Recent systems such as DeepSeek-V3 also use MTP-style modules for efficient generation (DeepSeek-AI, 2024).

3 Method

3.1 LLM2Vec Encoder

The encoder is a decoder-only LLM adapted for bidirectional encoding. This follows LLM2Vec (BehnamGhader et al., 2024). The encoder reads the source article and produces contextual token states. In the Llama version, the encoder

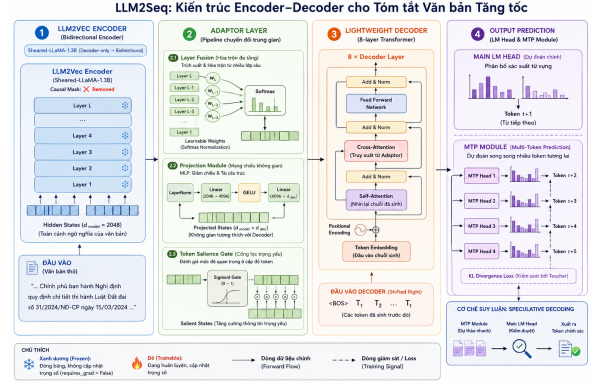


Figure 1: LLM2Seq turns a decoder-only LLM into the encoder side of a summarization system. A gated adaptor connects the encoder memory to a small decoder.

is LLM2Vec-Sheared-LLaMA-mntp. In the Qwen version, the encoder is Qwen3-Embedding-0.6B.

The WikiLingua setting uses long inputs. The main configs use 3,072 source tokens and 384 target tokens. This makes full fine-tuning expensive. Phase 2 therefore uses LoRA on the encoder (Hu et al., 2022).

3.2 Gated Residual Adaptor

The adaptor is the bridge between encoder and decoder. It has five parts.

First, token-wise layer fusion combines selected encoder layers. This is useful because shallow and deep Transformer layers store different information (Wang et al., 2019). Second, a gated residual projection maps the encoder dimension into the decoder dimension. Third, a salience gate filters token-level noise. Fourth, an EncStack refines the memory before cross-attention. Fifth, 16 learned global memory tokens are prepended to the encoder memory.

This design keeps the bridge expressive. A single projection is simple, but it can throw away source information. For summarization, that loss is costly.

3.3 Lightweight Decoder

The decoder is an 8-layer Transformer decoder. It has hidden size 1024, 16 heads, and a 4096-dimensional feed-forward layer. It uses masked self-attention for target tokens and cross-attention over the adapted source memory. The decoder is smaller than the encoder. This lets the encoder do most of the reading work while the decoder focuses on generation.

3.4 MTP and Self-Distillation

Phase 3 adds cascaded MTP heads. We use $K = 4$ draft heads. The main decoder path is frozen. Only the MTP module is trained. The loss is:

$$\mathcal{L} = \lambda_{CE}\mathcal{L}_{CE} + \lambda_{KL}\mathcal{L}_{KL}.$$

The current setup uses $\lambda_{CE} = 0.6$ and $\lambda_{KL} = 0.2$. The KL term starts at 20% of training and ramps until 50%. This gives the draft heads time to learn hard labels before matching the teacher distribution. The MTP head weights are $[1.0, 0.8, 0.5, 0.25]$.

At inference time, MTP proposes several tokens. The main head verifies them. Accepted tokens are emitted together. If too few tokens are accepted, MTP can lose to normal autoregressive decoding because verification adds overhead.

3.5 Training Stages

LLM2Seq is trained in three stages. Phase 1 freezes the encoder and trains the adaptor, decoder, and LM head. Phase 2 resumes from Phase 1 and trains encoder LoRA, adaptor, decoder, and LM head. Phase 3 resumes from Phase 2, freezes the main summarizer, and trains only the MTP blocks.

This order is deliberate. The MTP teacher should be stable before the draft heads learn from it.

4 Experiments

4.1 Dataset

We evaluate on WikiLingua, a multilingual abstractive summarization dataset built from WikiHow articles (Ladhak et al., 2020). The project uses the English split. After preprocessing, there are 13,999 training examples, 1,680 validation examples, and 3,901 test examples. One empty test row is removed.

4.2 Baselines

We compare three systems:

- **LLM2Seq-Llama:** LLM2Vec-Sheared-LLaMA encoder plus LLM2Seq decoder.
- **LLM2Seq-Qwen:** Qwen3-Embedding-0.6B encoder plus LLM2Seq decoder.
- **T5Gemma:** T5Gemma-2-1B-1B fine-tuned with LoRA.

All systems use greedy decoding. The generation configs use no sampling, a repetition penalty, and a no-repeat trigram constraint.

4.3 Metrics

We use ROUGE for summarization quality (Lin, 2004). We also report BLEU with standard reporting practice in mind (Post, 2018), chrF, mean prediction length, too-long rate, and measured decoding speed. For speed, we report generated tokens per second and mean latency from the project evaluation logs.

5 Results

5.1 Main Summarization Results

Table 1 shows the main WikiLingua test results. Qwen is much better than Llama inside the same LLM2Seq design. T5Gemma is strongest on ROUGE-2 and ROUGE-L, but it is also much more verbose.

The comparison tells a clear story. The Llama encoder under-generates. Its mean summary length is only 29.10 words, while the reference mean is 51.86. The Qwen encoder produces longer and more useful summaries. It raises ROUGE-1 by 5.65 points and ROUGE-L by 2.77 points over Llama. T5Gemma gives the best overlap scores, but its summaries are much longer than the references.

5.2 Decoding Speed

Table 2 shows throughput for the base autoregressive model. T5Gemma is fastest in this evaluation setup. LLM2Seq-Qwen is slower than T5Gemma but uses a smaller encoder-side model and produces shorter summaries.

5.3 MTP Results

MTP is tested with verified decoding. The goal is to reduce the number of serial decoding steps. Table 3 shows that MTP behaves differently for Llama and Qwen.

For Llama, MTP accepts more tokens per step but still slows down. This is a negative result. The verifier costs more than the saved steps.

For Qwen, MTP gives a real speedup. The measured token-per-second speedup is 1.17x. Mean latency improves from 0.779s to 0.662s. P95 latency improves from 1.142s to 0.922s. Quality changes are small: ROUGE-1 drops by 0.11 and ROUGE-L drops by 0.19.

Model	R-1	R-2	R-L	BLEU	chrF	Pred. words	Too-long
LLM2Seq-Llama	48.36	15.54	29.05	3.43	15.45	29.10	5.46
LLM2Seq-Qwen	54.01	20.83	31.83	7.58	24.66	58.31	35.79
T5Gemma-2-1B-1B	54.24	27.42	33.89	8.84	33.60	90.89	68.03

Table 1: WikiLingua test results. Scores are from the project result files. Too-long is a percentage.

Model	tok/s	Mean latency
LLM2Seq-Llama	114.74	0.697s
LLM2Seq-Qwen	95.87	0.779s
T5Gemma-2-1B-1B	257.70	0.412s

Table 2: Autoregressive decoding speed on the WikiLingua test set.

5.4 Adaptor Analysis

The full adaptor is used in all reported LLM2Seq runs. It includes layer fusion, gated residual projection, salience gating, EncStack, and global memory tokens. The current project snapshot does not include separate completed metrics for removing each component. We therefore treat the adaptor ablation as future controlled work, not as a claimed result.

Even without per-component ablation, the main comparison is useful. With the same adaptor and decoder design, changing the encoder from Llama to Qwen gives a large improvement. This suggests that the encoder representation is a major driver of summarization quality.

6 Discussion

Why Qwen helps. The Qwen encoder likely gives cleaner sentence-level and document-level states for WikiLingua. The adaptor can only pass on information that exists in the encoder. When the encoder is stronger for this data, the decoder receives better memory. This is why the Qwen variant closes much of the gap to T5Gemma.

Why T5Gemma still wins some metrics. T5Gemma is a native encoder-decoder model. It also has a larger 1B-1B backbone. It produces longer summaries, which helps overlap metrics such as ROUGE-2. But the long output is not always a pure win. Its too-long rate is 68.03%, while LLM2Seq-Qwen is 35.79%.

Why MTP speedup is limited. MTP reduces the number of decoding steps. For Qwen, it emits 2.30 tokens per step on average. But the measured speedup is only 1.17x. The gap comes from veri-

fier overhead and Python-level control flow. This matches the general lesson from speculative decoding: fewer steps are useful only when verification is cheap (Leviathan et al., 2023). Future work should move verification into a fused kernel, `torch.compile`, or a runtime such as vLLM. Quantization methods such as SmoothQuant may also help deployment under memory limits (Xiao et al., 2023).

7 Conclusion

LLM2Seq shows that decoder-only LLMs can be reused for abstractive summarization in an encoder-decoder system. The Qwen encoder version is close to T5Gemma on ROUGE-1 and produces more compact summaries. MTP gives a measured speedup for Qwen with small quality loss, but it fails for Llama under the same idea.

The main lesson is practical. Architecture matters, but implementation cost also matters. For summarization, a good encoder and a careful adaptor are needed first. MTP should come later, after the main summarizer is stable and the verifier is efficient.

Limitations

This report uses the completed metrics available in the project result files. It does not claim a full per-component adaptor ablation because those logs are not present in the current snapshot. The MTP speed numbers also measure the current Python implementation, not a fused serving runtime.

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*.
- Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large language models are secretly powerful text encoders. In *First Conference on Language Modeling*.

Model	Mode	tok/s	Speedup	Tokens/step	R-1	R-L
Llama	AR	114.74	1.00x	1.00	48.36	29.05
Llama	MTP	73.14	0.64x	2.45	48.36	29.05
Qwen	AR	95.87	1.00x	1.00	54.01	31.83
Qwen	MTP	111.71	1.17x	2.30	53.90	31.64

Table 3: Verified MTP results. AR means autoregressive decoding.

- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.
- DeepSeek-AI. 2024. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better and faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Faisal Ladhak, Esin Durmus, Claire Cardie, and Kathleen McKeown. 2020. WikiLingua: A new benchmark dataset for cross-lingual abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4034–4048.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*.
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880.
- Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81.
- Yingfeng Luo, Tong Zheng, Yongyu Mu, Bei Li, Qinghong Zhang, Yongqi Gao, Ziqiang Xu, Peinan Feng, Xiaoqian Liu, Tong Xiao, and Jingbo Zhu. 2025. Beyond decoder-only: Large language models can be good encoders for machine translation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 9399–9431.
- Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2024. Generative representational instruction tuning. *arXiv preprint arXiv:2402.09906*.
- Matt Post. 2018. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 186–191.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems*.
- Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.
- Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. 2019. Learning deep transformer models for machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1810–1822.

- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*.
- Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, and 1 others. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.
- Biao Zhang, Paul Suganthan, Gael Liu, Ilya Philippov, Sahil Dua, Ben Hora, Kat Black, Gus Martins, Omar Sanseviero, Shreya Pathak, Cassidy Hardin, Francesco Visin, Jiageng Zhang, Kathleen Kenealy, Qin Yin, Olivier Lacombe, Armand Joulin, Tris Warkentin, and Adam Roberts. 2025. T5Gemma 2: Seeing, reading, and understanding longer. *arXiv preprint arXiv:2512.14856*.
- Guoliang Zhao, Ruobing Xie, An Wang, Shuaipeng Li, Huaibing Xie, and Xingwu Sun. 2026. Self-distillation for multi-token prediction. *arXiv preprint arXiv:2603.23911*.